Assignment-3: Test Planning and Test Case Design

Software Testing and Quality Assurance • MCA 2nd Year

Assignment 3 — Answers

Q1. What is Test Planning? Explain the main steps involved in the test planning process.

Test planning is the systematic process of deciding what will be tested, how testing will be performed, who will perform it, what resources are required, and when activities will take place. It provides a roadmap for testing and helps control cost, time, and quality.

Main steps are: (1) analyse requirements and identify testable features; (2) define test objectives, scope, assumptions, and exclusions; (3) select test levels and techniques; (4) identify people, devices, tools, and environments; (5) estimate effort and cost; (6) prepare a test schedule and milestones; (7) define entry and exit criteria; (8) identify risks and contingency actions; (9) plan test data, deliverables, and defect reporting; and (10) review, approve, and update the plan as the project changes.

For example, for an online shopping application, the plan may cover login, product search, cart, payment, and order tracking, with separate dates and responsible testers for each area.

Q2. What is a Test Strategy? Explain its importance in software testing.

A test strategy is a high-level approach that describes the principles, methods, levels, and types of testing to be used in a project or organisation. It explains how quality risks will be addressed and how testing effort will be prioritised.

It is important because it creates a consistent approach, aligns testing with business risks, guides the choice of manual and automated testing, clarifies the test levels and environments, supports realistic resource planning, and helps teams communicate. A good strategy also defines how results and defects will be evaluated.

For example, a banking application may prioritise security, transaction accuracy, recovery, and performance, while a content website may place greater emphasis on usability, browser compatibility, and availability.

Q3. What is a Test Plan? Explain the major contents of a Test Plan.

A test plan is a documented description of the scope, objectives, resources, schedule, and activities for a particular testing effort. It converts the overall testing approach into actionable work.

Major contents include: project and document details; objectives; features to be tested and not tested; test scope; test approach and levels; test environment and tools; test data requirements; roles and responsibilities; schedule and milestones; effort and cost estimates; entry and exit criteria; deliverables; defect recording and reporting process; risks and mitigation; dependencies; and review/approval details.

A test plan is useful because it makes responsibilities clear, sets expectations, and provides a basis for tracking progress. It should be updated when requirements, deadlines, or project risks change.

Q4. Differentiate between a Test Strategy and a Test Plan.

A test strategy is generally high-level and explains the overall approach to testing. A test plan is more detailed and explains how that approach will be carried out for a particular project, release, or test phase.

Key differences:
• Purpose: Strategy defines the testing principles and approach; plan defines the tasks and execution arrangements.
• Scope: Strategy may apply across projects or to a whole project; plan is usually specific to a project, release, or phase.
• Content: Strategy covers test levels, techniques, risk approach, automation principles, and quality goals; plan covers features, people, schedule, environment, deliverables, and entry/exit criteria.

• Change: Strategy changes less frequently; a plan may be revised as dates, resources, or scope change.
• Relationship: The plan should follow and implement the strategy.

Example: the strategy may require risk-based testing and automation of regression tests. The test plan specifies which regression cases will be automated, by whom, and by what date.

Q5. What are the main goals and objectives of software testing? Explain.

The main goal of software testing is to evaluate software and provide information about its quality and risks before release. Testing can reveal defects, but it cannot prove that a complex system is completely defect-free.

Important objectives are: (1) detect defects before users encounter them; (2) verify that requirements and specifications are met; (3) validate that the product meets user needs; (4) assess reliability, performance, security, usability, compatibility, and recovery; (5) prevent failures that could cause data loss or financial harm; (6) provide stakeholders with evidence for release decisions; and (7) reduce the cost of fixing defects by finding them early.

For example, testing an online payment feature checks that valid payments succeed, invalid transactions are rejected safely, and the system does not charge a customer twice.

Q6. What are Test Phases? Explain the different phases of software testing.

Test phases are stages in the testing process, each with a different purpose. Common phases include:

1. Requirement analysis: review requirements, identify ambiguities, test conditions, and risks.
2. Test planning: determine scope, strategy, resources, schedule, and completion criteria.
3. Test design and development: prepare test cases, test scripts, test data, and traceability links.
4. Environment setup: configure hardware, software, accounts, networks, and test data.
5. Test execution: run tests, compare actual results with expected results, and record evidence.
6. Defect reporting and retesting: log defects, support fixes, and verify corrections.
7. Regression testing: rerun relevant tests to check that changes have not damaged existing functions.
8. Test closure: evaluate exit criteria, prepare a summary report, record lessons learned, and archive artefacts.

These phases can overlap and may be repeated in iterative or Agile development.

Q7. What factors should be considered while developing a Test Strategy?

A test strategy should consider business goals, user expectations, and the consequences of failure. Important factors include requirements and their stability; application size and complexity; technical architecture; regulatory and security obligations; product risks; test levels and test types; manual versus automated testing; available tools and environments; test data and privacy; team skills and availability; schedule and budget; integration dependencies; supported platforms and browsers; defect handling; and measurable exit criteria.

Risk-based prioritisation is especially useful: functions with high business impact or a high probability of failure receive more testing. The strategy should be reviewed when major requirements or risks change.

Q8. Explain the resource requirements needed for software testing.

Testing resources are the people, equipment, tools, environments, time, and information required to carry out testing.

• Human resources: test manager, test analysts, manual testers, automation engineers, developers for defect fixes, domain experts, and security/performance specialists where needed.
• Hardware and infrastructure: computers, mobile devices, servers, networks, cloud resources, and test labs.
• Software and tools: test management systems, defect trackers, automation frameworks, performance tools, and monitoring utilities.
• Test environment: configured operating systems, databases, browsers, APIs, and integrations that resemble the target environment.
• Test data: valid, invalid, boundary, and realistic data, prepared safely with privacy protection.
• Time and budget: effort for planning, design, execution, defect investigation, retesting, regression, and reporting.
• Documentation: requirements, designs, user stories, acceptance criteria, and previous defect records.

Resource planning prevents delays and helps ensure that testing is practical and sufficient.

Q9. What is a Test Schedule? Explain the importance of scheduling in software testing.

A test schedule is a timeline that lists testing activities, their start and finish dates, dependencies, milestones, and responsible people. It may include requirement review, test design, environment setup, execution, retesting, regression, and closure.

Scheduling is important because it coordinates testing with development and release dates, makes workload visible, identifies dependencies, helps allocate people and equipment, provides milestones for progress tracking, and highlights delays early. A realistic schedule includes time for defect fixes and retesting rather than assuming that the first test run will pass.

For example, payment testing cannot be completed until the payment gateway test environment is available. This dependency should be reflected in the schedule.

Q10. What is a Test Case? Explain the main components of a test case.

A test case is a set of conditions, inputs, actions, and expected results used to check whether a particular function behaves as required.

Main components are: test case ID; title or objective; requirement reference; preconditions; test environment; input/test data; execution steps; expected result; actual result; pass/fail status; priority; severity or risk where relevant; postconditions; and comments or evidence such as screenshots and logs.

Example: for login, enter a registered username and correct password, select Login, and expect the user to reach the account page. A clear test case is repeatable and gives the tester an objective basis for deciding whether the result passes.

Q11. Explain the process of designing and writing effective test cases.

Effective test-case design begins by understanding requirements, user stories, acceptance criteria, and business risks. Next, identify test conditions and divide the feature into normal, invalid, boundary, and exceptional situations. Choose suitable techniques such as equivalence partitioning, boundary value analysis, decision tables, or state-transition testing.

For each case, write a clear objective, preconditions, data, numbered steps, and measurable expected results. Include positive and negative cases, permission checks, error handling, and relevant compatibility or performance concerns. Link cases to requirements for traceability, review them with peers, remove duplication, and prioritise them by risk. Finally, execute the cases, record actual results, and maintain them when the application changes.

A good case is clear, independent where possible, repeatable, and specific enough that another tester can follow it.

Q12. What is Bug Reporting? Explain the important information included in a bug report.

Bug reporting is the process of documenting an observed difference between actual and expected software behaviour so that the team can investigate and correct it.

A useful bug report includes a unique ID, concise title, application/build version, environment, preconditions, reproducible steps, expected result, actual result, reproducibility rate, severity, priority, screenshots or logs, reporter, date, and related requirement or test case. It may also include owner, status, root cause, fix version, and retest result as the defect progresses.

Reports should be factual, clear, and reproducible. For example, “Checkout fails” is vague; “After selecting UPI and confirming payment, the page displays a blank screen on build 1.4” is more actionable when supported by steps and logs.

Q13. Explain the Bug Life Cycle in software testing.

The bug life cycle describes the states a bug passes through from discovery to closure. A typical flow is:

New → Assigned → Open/In Progress → Fixed → Retest → Verified → Closed.

If the fix fails, the defect is Reopened and returned to the developer. A report may also be marked Rejected/Invalid if it is not a defect, Duplicate if it already exists, Deferred if it will be handled later, or Cannot Reproduce if the team cannot reproduce it with the supplied information. Exact names vary by organisation and tool.

The tester logs evidence, the triage team prioritises and assigns the issue, the developer investigates and fixes it, and the tester retests the build. Once the correction is verified and relevant regression checks pass, the defect is closed.

Q14. What are Test Metrics? Explain their importance in measuring testing activities.

Test metrics are quantitative measurements used to understand testing progress, coverage, quality, and efficiency. Examples include number of test cases planned and executed, pass/fail rate, requirements coverage, defects found, defect leakage, and average time to resolve defects.

Metrics help managers compare progress with the plan, identify areas with insufficient coverage, estimate remaining effort, track defect trends, and communicate status using evidence. They can also support process improvement by showing recurring bottlenecks.

Metrics must be interpreted in context. A high number of defects does not automatically mean poor testing; it may reflect complex or poor-quality software. Metrics should support judgement rather than encourage testers to maximise test counts or hide defects.

Q15. Explain different types of testing metrics used in software testing.

Common testing metrics include:

1. Test progress metrics: planned, designed, executed, passed, failed, and blocked test cases.
2. Test coverage metrics: percentage of requirements, features, risks, or code covered by tests.
3. Defect metrics: defects by severity, defect density, open/closed defects, defect arrival rate, and defect leakage to production.
4. Execution metrics: pass rate, failure rate, blocked rate, and test execution speed.
5. Productivity metrics: test cases designed or executed per person or time period, interpreted alongside complexity and quality.
6. Efficiency metrics: time and effort spent per activity, automation execution time, and defect turnaround time.
7. Reliability/release metrics: production incidents, failure rate, and escaped defects after release.
8. Automation metrics: percentage of suitable tests automated, automation pass rate, and maintenance effort.

No single metric gives a complete picture; several measures should be reviewed together.

Q16. What are Test Case Design Techniques? Explain their importance in software testing.

Test case design techniques are systematic methods for selecting test inputs and conditions so that tests find useful defects without unnecessary duplication.

Black-box techniques include equivalence partitioning, boundary value analysis, decision-table testing, state-transition testing, and use-case testing. White-box techniques include statement, branch, and condition coverage. Experience-based techniques include exploratory testing and error guessing.

These techniques improve coverage, help test normal and exceptional behaviour, reduce redundant cases, and make test design more systematic. For example, boundary value analysis focuses on values at the edges of an allowed range, where mistakes are common. The technique selected depends on the requirement, risk, and available information.

Q17. What is Equivalence Partitioning? Explain with a suitable example.

Equivalence partitioning divides possible inputs into groups expected to be handled in the same way. A representative value from each group is tested, reducing the number of test cases while retaining useful coverage.

Example: suppose an age field accepts whole numbers from 18 through 60. The partitions are: invalid ages below 18; valid ages from 18 to 60; and invalid ages above 60. Representative tests might be 15, 35, and 65. Additional checks may be needed for non-numeric input, blank values, decimals, or negative values, depending on the specification.

The technique assumes that values within a partition are treated similarly. It is efficient, but it may miss errors at the boundaries, so it is often combined with boundary value analysis.

Q18. What is Boundary Value Analysis? Explain with a suitable example.

Boundary value analysis (BVA) focuses on values at the edges of an input range because errors frequently occur at minimum and maximum limits.

Example: if an age field accepts whole numbers from 18 to 60 inclusive, useful boundary tests are 17 (just below minimum), 18 (minimum), 19 (just above minimum), 59 (just below maximum), 60 (maximum), and 61 (just above maximum). Expected results are rejection for 17 and 61, and acceptance for 18, 19, 59, and 60, assuming all other conditions are valid.

BVA is effective for numeric ranges, lengths, dates, and limits. The exact test values depend on whether endpoints are inclusive and on the data type and specification.

Q19. Differentiate between Equivalence Partitioning and Boundary Value Analysis.

Equivalence Partitioning (EP) divides input data into groups expected to behave similarly and selects representative values from each group. Boundary Value Analysis (BVA) selects values at, just below, and just above the edges of an allowed range.

- Focus: EP focuses on classes or partitions; BVA focuses on range limits.
- Test selection: EP may select one representative per class; BVA selects edge values.
- Best use: EP is useful for categories and broad input domains; BVA is especially useful for ordered ranges and limits.
- Example for age 18–60: EP tests could be 15, 35, and 65; BVA tests could be 17, 18, 19, 59, 60, and 61.
- Benefit: EP reduces redundant testing; BVA targets common off-by-one and limit errors.

They complement one another and are often used together.

Q20. What is Test Case Planning? Explain the steps involved in test case planning.

Test case planning determines which test cases are needed, how they will be organised, and how they will cover requirements and risks. It connects the test scope to executable checks.

Steps include: (1) study requirements, acceptance criteria, and design; (2) identify features and test conditions; (3) assess risks and prioritise critical functions; (4) select test design techniques; (5) define test cases and expected results; (6) prepare test data and preconditions; (7) map cases to requirements using a traceability matrix; (8) estimate effort and assign responsibilities; (9) review cases for clarity, completeness, and duplication; and (10) schedule execution and define how results and defects will be recorded.

The plan should be maintained as requirements change. Good test case planning makes coverage visible and reduces the chance of missing important scenarios.